

12. 車体物理・電気モデル本体

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	4
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	5
5.3 import 群の詳細	5
6 このファイルを読む前に必要な基礎知識	6
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	6
6.2 名前、代入、型、参照	6
6.3 関数、引数、戻り値、スコープ	6
6.4 module、package、import、console_scripts	7
6.5 例外、try/finally、with、資源解放	7
6.6 class、独自クラス、インスタンス、self、継承	7
6.7 先頭アンダースコア、__init__、名前付けの作法	8
6.8 デコレータと @dataclass	8
6.9 list、dict、deque、NumPy 配列、pandas DataFrame	9
6.10 車両力学、電力収支、効率 map	9
6.11 SoC、内部抵抗、端子電圧、温度、MHE	9
6.12 freshness、filter、guard、fallback、fail-safe	9
6.13 制御、状態、入力、モデル予測制御 MPC	9
6.14 CSV/YAML の data contract と検証	10

7 関数・クラスを上から順に解説	10
7.1 L10 関数 <code>_is_symbolic</code>	10
7.2 L17 クラス <code>Params</code>	11
7.3 L73 クラス <code>SolarCarModel</code>	12
7.4 L74 関数 <code>SolarCarModel.__init__</code>	14
7.5 L124 関数 <code>SolarCarModel.eff_drive</code>	16
7.6 L137 関数 <code>SolarCarModel.eff_regen</code>	17
7.7 L150 関数 <code>SolarCarModel._update_mode_limits</code>	19
7.8 L158 関数 <code>SolarCarModel._select_mode</code>	20
7.9 L169 関数 <code>SolarCarModel.select_drive_mode</code>	21
7.10 L172 関数 <code>SolarCarModel.R_int</code>	21
7.11 L179 関数 <code>SolarCarModel.pv_balance</code>	22
7.12 L222 関数 <code>SolarCarModel.pv_power_mppt</code>	24
7.13 L225 関数 <code>SolarCarModel._scaled_slope_pct</code>	25
7.14 L228 関数 <code>SolarCarModel.charge_efficiency</code>	25
7.15 L235 関数 <code>SolarCarModel.soc_step</code>	26
7.16 L262 関数 <code>SolarCarModel.ocv_from_soc</code>	28
7.17 L271 関数 <code>SolarCarModel.load_ocv_map</code>	29
7.18 L280 関数 <code>SolarCarModel.air_density</code>	30
7.19 L297 関数 <code>SolarCarModel.resistive_forces</code>	31
7.20 L344 関数 <code>SolarCarModel.battery_iv</code>	33
7.21 L368 関数 <code>SolarCarModel.mech_power</code>	34
7.22 L389 関数 <code>SolarCarModel.auxiliary_power</code>	35
7.23 L396 関数 <code>SolarCarModel.scheduled_auxiliary_power</code>	36
7.24 L405 関数 <code>SolarCarModel.torque_from_mech</code>	37
7.25 L416 関数 <code>SolarCarModel.electrical_balance</code>	38
8 処理の流れ	41
9 このファイルの位置づけ	41

1 対象

項目	内容
ファイル	mpc_solarcar/model.py
ソース SHA-256	91a29cea04d184a73105566c514d824c8259be3a801df05ddfd82af7e8c863c1
種別	Python
区分	model
役割	空力、転がり、坂、PV、MPPT、drive/regen 効率、battery IV、SoC 更新を統合した vehicle model。
起動文脈	planner と simulation の数理コア。

2 このファイルを読む理由

空力、転がり、坂、PV、MPPT、drive/regen 効率、battery IV、SoC 更新を統合した vehicle model。

- SolarCarModel と Params が中心。
- electrical_balance が planner 側で最も多く呼ばれる。
- resistive_forces、battery_iv、soc_step が各所から再利用される。

3 呼び出し関係

3.1 どこから呼ばれるか

- mpc_node.py
- solar_state_node.py
- scripts/solar_sim.py
- estimator.py

3.2 次に読むと理解が進むファイル

- mpc_solarcar/utils_maps.py

3.3 同一リポジトリ内の主要依存

- mpc_solarcar/utils_maps.py

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
)

@dataclass
class Params:
    dt: float=600.0
    rho: float=1.18
    air_density_mode: str='constant'
    air_density_reference_pressure_pa: float=101325.0
    CdA: float=0.13
    Crr: float=0.002
    Crr_per_wheel: float=0.0
    m: float=250.0
    g: float=9.80665
    P_aux: float=60.0
    P_aux_stopped: float=60.0
    P_aux_night: float=0.0
    aux_night_ghi_threshold_wm2: float=20.0
    gear_eta: float=0.98
    gear_ratio: float=6.0
    wheel_radius: float=0.28
    wheel_count: int=4
    driven_wheel_count: int=2
```

5 主要構造の読み取り

主要クラスは Params, SolarCarModel。主要関数は eff_drive, eff_regen, select_drive_mode, R_int, pv_balance, pv_power_mppt, charge_efficiency, soc_step。

5.1 主要クラス・関数

行	種別	名前	補足
17	class	Params	
73	class	SolarCarModel	
10	function	_is_symbolic	args: x
74	function	__init__	args: self, drive_map_path, regen_map_path, Rint_map_path, params, panel_eff_map_path, mppt_eff_map_path, drive_map_eco_path, drive_map_power_path, regen_map_eco_path, regen_map_power_path, ocv_soc_map_path
124	function	eff_drive	args: self, v_ms, tau_nm
137	function	eff_regen	args: self, v_ms, tau_nm
150	function	_update_mode_limits	args: self
158	function	_select_mode	args: self, v_ms, tau_nm
169	function	select_drive_mode	args: self, v_ms, tau_nm
172	function	R_int	args: self, T_C, z
179	function	pv_balance	args: self, G_poa, T_cell_C
222	function	pv_power_mppt	args: self, G_poa, T_cell_C
225	function	_scaled_slope_pct	args: self, slope_pct
228	function	charge_efficiency	args: self, P_pack

235	function	soc_step	args: self, z, P_pack, dt_sec
262	function	ocv_from_soc	args: self, z
271	function	load_ocv_map	args: self, path
280	function	air_density	args: self, ambient_temp_c, elevation_m
297	function	resistive_forces	args: self, v_ms, slope_pct, headwind_ms
344	function	battery_iv	args: self, P_pack, z, Tbat_C
368	function	mech_power	args: self, v_ms, slope_pct, headwind_ms, inertial_power_w
389	function	auxiliary_power	args: self, aux_power_w
396	function	scheduled_auxiliary_power	args: self
405	function	torque_from_mech	args: self, P_mech, v_ms, wheel_radius, ratio
416	function	electrical_balance	args: self, v_ms, slope_pct, z, Tbat_C, G_poa, Tcell_C, headwind_ms, aux_power_w, inertial_power_w, ambient_temp_c, elevation_m

5.2 ファイルを上から読んだときの定義順

行	内容
3	例外処理を伴う try ブロックを実行する。
10	関数 <code>_is_symbolic</code> を定義する。
17	クラス <code>Params</code> を定義する。
73	クラス <code>SolarCarModel</code> を定義する。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
1	<code>import math</code>	Python 標準のスカラ・数値関数と定数を使うため。実際に参照する名前と行は使用位置欄で確認する。このファイル内での主な使用位置は L329, L331, L336, L362。
2	<code>import numpy as np</code>	数値配列、clip、補間、統計計算を行うため。このファイル内での主な使用位置は L135, L148, L266, L269, L292, L347, L447。
4	<code>import casadi as ca</code>	casadi モジュールを利用するため。このファイル内での主な使用位置は L6, L11, L12, L126, L128, L130, L139, L143, ...。
7	<code>from dataclasses import dataclass</code>	<code>dataclasses</code> から <code>dataclass</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L16。
8	<code>from utils_maps import read_eff_map, read_Rint_map, read_map, bilinear_interp, read_1d_map</code>	効率マップ・抵抗マップ読込補間から <code>read_eff_map</code> , <code>read_Rint_map</code> , <code>read_map</code> , <code>bilinear_interp</code> , <code>read_1d_map</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/utils_maps.py</code> 。このファイル内での主な使用位置は L81, L82, L93, L95, L97, L99, L101, L106, ...。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Pythonインタプリタが読み込む -> OS上のプロセス -> Pythonオブジェクトをメモリに生成 -> 関数やcallbackを実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# nodeとaliasは同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):
    return min(maximum, max(minimum, value))

v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが ‘self.z’ を更新すればオブジェクトの状態が残るが、単に ‘z’ を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の ‘console_scripts’は、端末で使う実行可能名と ‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->
main()を呼ぶ
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 class、独自クラス、インスタンス、self、継承

クラスは、保持するデータと、そのデータを扱う関数を一つの型としてまとめる設計単位である。「独自クラス」とは Python や ROS 2 が最初から用意した型ではなく、このプロジェクトが目的に合わせて定義した新しい型を指す。

‘class MPCNode(Node)’は、rclpy の Node を基底クラスとして継承し、Node が持つ publisher、subscription、timer、parameter などの機能に MPC 固有機能を追加する。丸括弧内は関数引数ではなく基底クラス指定である。

‘MPCNode()’はクラスオブジェクトを呼び出して新しいインスタンスを作る。Python は新しい実体を用意し、その実体を第 1 引数 self として ‘__init__’へ渡す。‘self’は予約語ではなく慣習的な名前だが、この慣習を守ることでコードの意味が共有される。

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1
```

```
counter = Counter()
counter.increment()
```

‘`super().__init__(...)`’は継承元の初期化処理を呼ぶ。MPCNode ではこれを省くと ROS ノードとして必要な内部実体が作られず、‘`create_publisher`’などを正しく使えない。

根拠資料

- [Python 公式チュートリアル: Classes](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.7 先頭アンダースコア、`__init__`、名前付けの作法

‘`self.step_solar`’の先頭 1 個のアンダースコアは「クラス内部で使う実装詳細」という慣習であり、アクセスを強制禁止する機構ではない。外部コードから呼べるが、公開 API として依存しない意思表示である。

‘`__init__`’の前後 2 個のアンダースコアは Python が定めた特殊メソッド名である。クラス生成時に自動で呼ばれる。任意の名前に前後 2 個を付けて独自仕様を作ることは避ける。

‘`_`’で始まるローカル変数は、未使用または外部へ見せない意図を示す場合がある。例えば ‘`_base_csv`’は戻り値として受け取る必要はあるが、その後の処理では使わないことを読者へ示す。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.8 デコレータと `@dataclass`

‘`@名前`’は、直後に定義した関数またはクラスを別の関数へ渡し、その結果で定義名を置き換えるデコレータ構文である。‘`@Class`’という一般構文があるのではなく、‘`@dataclass`’など実際のデコレータ名を書く。

‘`@dataclass`’は型注釈付きフィールドから ‘`__init__`’、‘`__repr__`’、比較処理などを自動生成する。物理パラメータや解析結果のように、名前付きデータを一まとまりで運ぶ用途に向く。

```
from dataclasses import dataclass

@dataclass
class State:
    soc: float
    temperature_c: float

state = State(soc=0.8, temperature_c=30.0)
```

自動生成は検証を自動で保証しない。単位、許容範囲、相互依存制約は ‘`__post_init__`’ や別の検証関数で確認する必要がある。

根拠資料

- [Python 公式ライブラリ: dataclasses](#)
- [Python 公式チュートリアル: Classes](#)

6.9 list、dict、deque、NumPy 配列、pandas DataFrame

list は順序付き可変列、tuple は順序付きで通常変更しない列、dict はキーから値を引く対応表、deque は両端追加・削除が効率的なキューである。どの構造を選ぶかはアクセス方法と更新方法で決まる。

NumPy 配列は同種数値を連続的に扱い、要素ごとの演算、clip、補間、線形代数を簡潔に書く。shape は各次元の要素数であり、速度系列なら通常 '(N)」、候補集団なら '(population, N)となる。

pandas DataFrame は列名を持つ表である。CSV 読込後は列型、欠損、単位、timezone、並び順、重複を明示的に処理しなければ、数値計算が動いても意味が誤る。

6.10 車両力学、電力収支、効率 map

$$F_{\text{aero}} = \frac{1}{2} \rho C_d A (v + v_{\text{wind}})^2, \quad F_{\text{roll}} \approx mg C_{rr} \cos \theta, \quad F_{\text{grade}} = mg \sin \theta$$

車輪機械 power は概ね 'P_mech = F_total * v + P_inertia' で、駆動時と回生時で異なる効率 map を通して DC 側 power へ変換する。map は速度・torque などを軸にした実測または同定 table であり、範囲外の補間・clip 規則もモデルの一部である。

$$P_{\text{pack}} = P_{\text{drive,dc}} - P_{\text{regen,dc}} + P_{\text{aux}} - P_{\text{pv}}$$

符号規約は必ずコードで確認する。本プロジェクトでは正の pack power を放電側として SoC を減らす処理が中心であり、発電と回生は pack 負荷を下げる方向に働く。

6.11 SoC、内部抵抗、端子電圧、温度、MHE

$$V = V_{\text{oc}}(z, T) - IR_{\text{total}}(z, T, I), \quad z_{k+1} = z_k - \frac{\eta(I, T) I \Delta t}{Q_{\text{eff}}}$$

SoC は直接完全には観測できないため、電流積算、OCV、端子電圧、温度、容量、効率を組み合わせる。内部抵抗は SoC、温度、電流方向で変わり、発熱と電圧制約の両方へ影響する。

MHE は有限時間窓の状態と観測誤差をまとめて最適化し、SoC や温度を推定する。古い測定や欠損値を同じ重みで使うと推定が壊れるため、timestamp freshness と観測可用性を入口で確認する。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)

6.12 freshness、filter、guard、fallback、fail-safe

分散システムでは最後に受け取った値が現在も有効とは限らない。受信時刻と timeout から freshness を判定し、stale 値を計画状態へ無条件に同期しない。

filter は noise と一時的な飛び値を抑えるが、遅れを生む。slew limit は指令変化率を制限する。安全 guard は solver の cost 罰則とは別に、現在出力へ強制制約を適用する最後の防波堤である。

fallback は失敗時の代替動作を事前に決める設計である。前回計画保持、物理に基づく決定論的入力、停止、低速制限などから、故障 mode ごとに選ぶ。fallback 発生は status と log へ残し、正常解と区別する。

6.13 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を x 、操作入力を u 、外乱・予報を w とすると、離散モデルは ' $x[k+1] = f(x[k], u[k], w[k])$ ' と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが **receding horizon** である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)

6.14 CSV/YAML の data contract と検証

data contract は列名、型、単位、timezone、欠損可否、並び順、重複、許容範囲、先頭行、encoding を事前に決めた仕様である。単に CSV として読めることは、モデル入力として正しいことを意味しない。

同定用実測、map、route、forecast、stop、schedule はそれぞれ grain が異なる。生成時に schema validation、物理範囲、時間単調性、route 範囲、coverage を検査し、検査結果を artifact として残す。

学習用データと独立検証データを分離し、RMSE だけでなく bias、時系列残差、energy 積算誤差、終端 SoC、温度・電圧制約、外挿領域を評価する。

7 関数・クラスを上から順に解説

7.1 L10 関数 `_is_symbolic`

- 行範囲: L10–L14
- 所属: module 直下
- 定義: `_is_symbolic(x)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `hasattr`, `is_symbolic`, `isinstance`
- 戻り値の要点: `ca is not None and (isinstance(x, (ca.SX, ca.MX)) or (hasattr(x, 'is_symbolic') and x.is_symbolic()))`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `ca is not None and (isinstance(x, (ca.SX, ca.MX)) or (hasattr(x, 'is_symbolic') and x.is_symbolic()))` を返す。

代表コード断片

```
def _is_symbolic(x):  
    return ca is not None and (  
        isinstance(x, (ca.SX, ca.MX))  
        or (hasattr(x, 'is_symbolic') and x.is_symbolic())  
    )
```

7.2 L17 クラス Params

- 行範囲: L17–L71
- 所属: module 直下
- 定義: `Params(bases=None)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- `class` 文は新しい型を定義する。丸括弧内は継承する基底クラスである。
- `@` で始まる行は、定義したクラスを加工する decorator である。

上から順の処理

1. `dt` に 600.0 を代入する。
2. `rho` に 1.18 を代入する。
3. `air_density_mode` に 'constant' を代入する。
4. `air_density_reference_pressure_pa` に 101325.0 を代入する。
5. `CdA` に 0.13 を代入する。
6. `Crr` に 0.002 を代入する。
7. `Crr_per_wheel` に 0.0 を代入する。
8. `m` に 250.0 を代入する。
9. `g` に 9.80665 を代入する。
10. `P_aux` に 60.0 を代入する。
11. `P_aux_stopped` に 60.0 を代入する。
12. `P_aux_night` に 0.0 を代入する。

代表コード断片

```
class Params:
    dt: float=600.0
    rho: float=1.18
    air_density_mode: str='constant'
    air_density_reference_pressure_pa: float=101325.0
    CdA: float=0.13
    Crr: float=0.002
    Crr_per_wheel: float=0.0
    m: float=250.0
    g: float=9.80665
    P_aux: float=60.0
    P_aux_stopped: float=60.0
    P_aux_night: float=0.0
    aux_night_ghi_threshold_wm2: float=20.0
    gear_eta: float=0.98
    gear_ratio: float=6.0
    wheel_radius: float=0.28
    wheel_count: int=4
    driven_wheel_count: int=2
    motor_count: int=1
    motor_type: str='generic'
    inverter_eta: float=1.0
    pv_area: float=4.0
    pv_eta_ref: float=0.23
    pv_mu_p: float=-0.0045
    mppt_eta: float=0.95
    panel_gain: float=1.0
    # Calibration from the vehicle telemetry solar-power channel to actual
    # battery-side solar power. It is applied only at telemetry ingestion.
    solar_measurement_gain_to_pack: float=1.0
    # Aggregate battery-side limit of the installed MPPT channels. A value
    # at or below zero disables clipping for vehicles without declared data.
    pv_power_limit_w: float=0.0
    E_nom_Wh: float=3055.0
    Q_nom_Ah: float=0.0
    ...
```

7.3 L73 クラス SolarCarModel

- 行範囲: L73–L477
- 所属: module 直下
- 定義: SolarCarModel(bases=None)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- class 文は新しい型を定義する。丸括弧内は継承する基底クラスである。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 関数 `__init__` を定義する。
2. 関数 `eff_drive` を定義する。
3. 関数 `eff_regen` を定義する。
4. 関数 `_update_mode_limits` を定義する。
5. 関数 `_select_mode` を定義する。
6. 関数 `select_drive_mode` を定義する。
7. 関数 `R_int` を定義する。
8. 関数 `pv_balance` を定義する。
9. 関数 `pv_power_mppt` を定義する。
10. 関数 `_scaled_slope_pct` を定義する。
11. 関数 `charge_efficiency` を定義する。
12. 関数 `soc_step` を定義する。

代表コード断片

```
class SolarCarModel:
    def __init__(self, drive_map_path, regen_map_path, Rint_map_path,
                 params=None, panel_eff_map_path=None, mppt_eff_map_path=None,
                 drive_map_eco_path=None, drive_map_power_path=None,
                 regen_map_eco_path=None, regen_map_power_path=None,
                 ocv_soc_map_path=None):
        self.p = params or Params()
        self.aux_power_override_w = None
        self.v_grid, self.tau_grid, self.Z_drv = read_eff_map(drive_map_path)
        self.v_gridR, self.tau_gridR, self.Z_reg = read_eff_map(regen_map_path)
        self.drive_mode = 'auto'
        self.drive_mode_default = 'eco'
        self.drive_mode_tau_margin = 0.0
        self.maps_drive = {
            'default': (self.v_grid, self.tau_grid, self.Z_drv),
        }
        self.maps_regen = {
            'default': (self.v_gridR, self.tau_gridR, self.Z_reg),
        }
        if drive_map_eco_path:
            self.maps_drive['eco'] = read_eff_map(drive_map_eco_path)
        if drive_map_power_path:
            self.maps_drive['power'] = read_eff_map(drive_map_power_path)
        if regen_map_eco_path:
            self.maps_regen['eco'] = read_eff_map(regen_map_eco_path)
        if regen_map_power_path:
            self.maps_regen['power'] = read_eff_map(regen_map_power_path)
        self._update_mode_limits()
        self.Tg, self.zg, self.Rmap = read_Rint_map(Rint_map_path)
        self.panel_eff_map = None
```

```

self.mppt_eff_map = None
if panel_eff_map_path:
    try:
        self.Gg, self.Tcg, self.Z_panel = read_map(panel_eff_map_path)
        self.panel_eff_map = True
...

```

7.4 L74 関数 SolarCarModel.__init__

- 行範囲: L74–L122
- 所属: SolarCarModel
- 定義: `__init__(self, drive_map_path, regen_map_path, Rint_map_path, params=None, panel_eff_map_path=None, mppt_eff_map_path=None, drive_map_eco_path=None, drive_map_power_path=None, regen_map_eco_path=None, regen_map_power_path=None, ocv_soc_map_path=None)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Params`, `_update_mode_limits`, `read_ld_map`, `read_Rint_map`, `read_eff_map`, `read_map`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self.Z_drv`, `self.Z_reg`, `self._update_mode_limits`, `self.maps_drive`, `self.maps_regen`, `self.tau_grid`, `self.tau_gridR`, `self.v_grid`, `self.v_gridR`
- 更新する主なインスタンス属性: `self.Gg`, `self.Gm`, `self.Rmap`, `self.Tcg`, `self.Tg`, `self.Tm`, `self.Z_drv`, `self.Z_mppt`, `self.Z_panel`, `self.Z_reg`, `self.aux_power_override_w`, `self.drive_mode`, `self.drive_mode_default`, `self.drive_mode_tau_margin`, `self.maps_drive`, `self.maps_regen`, `self.mppt_eff_map`, `self.ocv_grid`, `self.ocv_soc_map`, `self.p`, `self.panel_eff_map`, `self.soc_grid`, `self.tau_grid`, `self.tau_gridR`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 7、ループ 0、`try` 3

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. `self.p` に `params` or `Params()` の結果を代入する。
2. `self.aux_power_override_w` に `None` の結果を代入する。
3. `(self.v_grid, self.tau_grid, self.Z_drv)` に `read_eff_map(drive_map_path)` の結果を代入する。
4. `(self.v_gridR, self.tau_gridR, self.Z_reg)` に `read_eff_map(regen_map_path)` の結果を代入する。
5. `self.drive_mode` に `'auto'` の結果を代入する。
6. `self.drive_mode_default` に `'eco'` の結果を代入する。
7. `self.drive_mode_tau_margin` に `0.0` の結果を代入する。

8. self.maps_drive に {'default': (self.v_grid, self.tau_grid, self.Z_drv)} の結果を代入する。
9. self.maps_regen に {'default': (self.v_gridR, self.tau_gridR, self.Z_reg)} の結果を代入する。
10. 条件 drive_map_eco_path を判定し、真なら内部処理を行う。
11. self.maps_drive['eco'] に read_eff_map(drive_map_eco_path) の結果を代入する。
12. 条件 drive_map_power_path を判定し、真なら内部処理を行う。
13. self.maps_drive['power'] に read_eff_map(drive_map_power_path) の結果を代入する。
14. 条件 regen_map_eco_path を判定し、真なら内部処理を行う。
15. self.maps_regen['eco'] に read_eff_map(regen_map_eco_path) の結果を代入する。
16. 条件 regen_map_power_path を判定し、真なら内部処理を行う。
17. self.maps_regen['power'] に read_eff_map(regen_map_power_path) の結果を代入する。
18. self._update_mode_limits(...) を実行する。
19. (self.Tg, self.zg, self.Rmap) に read_Rint_map(Rint_map_path) の結果を代入する。
20. self.panel_eff_map に None の結果を代入する。
21. self.mppt_eff_map に None の結果を代入する。
22. 条件 panel_eff_map_path を判定し、真なら内部処理を行う。
23. 例外処理を伴う try ブロックを実行する。
24. (self.Gg, self.Tcg, self.Z_panel) に read_map(panel_eff_map_path) の結果を代入する。
25. self.panel_eff_map に True の結果を代入する。
26. Exception を捕捉した場合:
27. self.panel_eff_map に None の結果を代入する。
28. 条件 mppt_eff_map_path を判定し、真なら内部処理を行う。
29. 例外処理を伴う try ブロックを実行する。
30. (self.Gm, self.Tm, self.Z_mppt) に read_map(mppt_eff_map_path) の結果を代入する。
31. self.mppt_eff_map に True の結果を代入する。
32. Exception を捕捉した場合:
33. self.mppt_eff_map に None の結果を代入する。
34. self.ocv_soc_map に None の結果を代入する。
35. 条件 ocv_soc_map_path を判定し、真なら内部処理を行う。
36. 例外処理を伴う try ブロックを実行する。
37. (self.soc_grid, self.ocv_grid) に read_ld_map(ocv_soc_map_path) の結果を代入する。
38. self.ocv_soc_map に True の結果を代入する。
39. Exception を捕捉した場合:
40. self.ocv_soc_map に None の結果を代入する。

代表コード断片

```
def __init__(self, drive_map_path, regen_map_path, Rint_map_path,
              params=None, panel_eff_map_path=None, mppt_eff_map_path=None,
              drive_map_eco_path=None, drive_map_power_path=None,
              regen_map_eco_path=None, regen_map_power_path=None,
              ocv_soc_map_path=None):
    self.p = params or Params()
    self.aux_power_override_w = None
    self.v_grid, self.tau_grid, self.Z_drv = read_eff_map(drive_map_path)
    self.v_gridR, self.tau_gridR, self.Z_reg = read_eff_map(regen_map_path)
    self.drive_mode = 'auto'
    self.drive_mode_default = 'eco'
    self.drive_mode_tau_margin = 0.0
    self.maps_drive = {
        'default': (self.v_grid, self.tau_grid, self.Z_drv),
    }
    self.maps_regen = {
        'default': (self.v_gridR, self.tau_gridR, self.Z_reg),
    }
    if drive_map_eco_path:
        self.maps_drive['eco'] = read_eff_map(drive_map_eco_path)
    if drive_map_power_path:
        self.maps_drive['power'] = read_eff_map(drive_map_power_path)
    if regen_map_eco_path:
        self.maps_regen['eco'] = read_eff_map(regen_map_eco_path)
    if regen_map_power_path:
        self.maps_regen['power'] = read_eff_map(regen_map_power_path)
    self._update_mode_limits()
    self.Tg, self.zg, self.Rmap = read_Rint_map(Rint_map_path)
    self.panel_eff_map = None
    self.mppt_eff_map = None
    if panel_eff_map_path:
        try:
            self.Gg, self.Tcg, self.Z_panel = read_map(panel_eff_map_path)
            self.panel_eff_map = True
        except Exception:
            ...
```

7.5 L124 関数 SolarCarModel.eff_drive

- 行範囲: L124–L135
- 所属: SolarCarModel
- 定義: `eff_drive(self, v_ms, tau_nm)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_is_symbolic`, `_select_mode`, `abs`, `bilinear_interp`, `clip`, `fabs`, `float`, `fmax`, `fmin`, `get`, `sqrt`
- 戻り値の要点: `float(np.clip(eff, 0.55, 0.99)) / ca.fmin(0.99, ca.fmax(0.55, eff))`
- 主なローカル代入名: `Z`, `eff`, `mode`, `t`, `tN`, `t_grid`, `v`, `vN`, `v_grid`
- 読み取る主なインスタンス属性: `self._select_mode`, `self.maps_drive`, `self.p`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. 条件 `_is_symbolic(v_ms) or _is_symbolic(tau_nm)` を判定し、真なら内部処理を行う。
2. `v` に `ca.fabs(v_ms)` の結果を代入する。
3. `t` に `ca.fabs(tau_nm)` の結果を代入する。
4. `vN` に `v / 35.0` の結果を代入する。
5. `tN` に `t / 60.0` の結果を代入する。
6. `eff` に `0.92 - 0.08 * vN * vN - 0.06 * ca.sqrt(tN + 1e-09)` の結果を代入する。
7. `eff` に `eff * float(self.p.drive_eff_scale)` の結果を代入する。
8. `ca.fmin(0.99, ca.fmax(0.55, eff))` を返す。
9. `mode` に `self._select_mode(float(v_ms), float(abs(tau_nm)))` の結果を代入する。
10. `(v_grid, t_grid, Z)` に `self.maps_drive.get(mode, self.maps_drive['default'])` の結果を代入する。
11. `eff` に `float(bilinear_interp(v_grid, t_grid, Z, float(v_ms), float(abs(tau_nm))))` の結果を代入する。
12. `eff` を `Mult` で更新する。
13. `float(np.clip(eff, 0.55, 0.99))` を返す。

代表コード断片

```
def eff_drive(self, v_ms, tau_nm):
    if _is_symbolic(v_ms) or _is_symbolic(tau_nm):
        v = ca.fabs(v_ms); t = ca.fabs(tau_nm)
        vN = v/35.0; tN = t/60.0
        eff = 0.92 - 0.08*vN*vN - 0.06*ca.sqrt(tN+1e-9)
        eff = eff * float(self.p.drive_eff_scale)
        return ca.fmin(0.99, ca.fmax(0.55, eff))
    mode = self._select_mode(float(v_ms), float(abs(tau_nm)))
    v_grid, t_grid, Z = self.maps_drive.get(mode, self.maps_drive['default'])
    eff = float(bilinear_interp(v_grid, t_grid, Z, float(v_ms), float(abs(tau_nm))))
    eff *= float(self.p.drive_eff_scale)
    return float(np.clip(eff, 0.55, 0.99))
```

7.6 L137 関数 `SolarCarModel.eff_regen`

- 行範囲: L137–L148
- 所属: `SolarCarModel`
- 定義: `eff_regen(self, v_ms, tau_nm)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_is_symbolic`, `_select_mode`, `abs`, `bilinear_interp`, `clip`, `fabs`, `float`, `fmax`, `fmin`, `get`
- 戻り値の要点: `float(np.clip(eff, 0.4, 0.95)) / ca.fmin(0.95, ca.fmax(0.4, eff))`
- 主なローカル代入名: `Z`, `eff`, `mode`, `t`, `tN`, `t_grid`, `v`, `vN`, `v_grid`

- 読み取る主なインスタンス属性: `self._select_mode`, `self.maps_regen`, `self.p`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. 条件 `_is_symbolic(v_ms) or _is_symbolic(tau_nm)` を判定し、真なら内部処理を行う。
2. `v` に `ca.fabs(v_ms)` の結果を代入する。
3. `t` に `ca.fabs(tau_nm)` の結果を代入する。
4. `vN` に `v / 35.0` の結果を代入する。
5. `tN` に `t / 60.0` の結果を代入する。
6. `eff` に `0.7 + 0.12 * vN - 0.05 * (tN - 0.3) * (tN - 0.3)` の結果を代入する。
7. `eff` に `eff * float(self.p.regen_eff_scale or self.p.drive_eff_scale)` の結果を代入する。
8. `ca.fmin(0.95, ca.fmax(0.4, eff))` を返す。
9. `mode` に `self._select_mode(float(v_ms), float(abs(tau_nm)))` の結果を代入する。
10. `(v_grid, t_grid, Z)` に `self.maps_regen.get(mode, self.maps_regen['default'])` の結果を代入する。
11. `eff` に `float(bilinear_interp(v_grid, t_grid, Z, float(v_ms), float(abs(tau_nm))))` の結果を代入する。
12. `eff` を `Mult` で更新する。
13. `float(np.clip(eff, 0.4, 0.95))` を返す。

代表コード断片

```
def eff_regen(self, v_ms, tau_nm):
    if _is_symbolic(v_ms) or _is_symbolic(tau_nm):
        v = ca.fabs(v_ms); t = ca.fabs(tau_nm)
        vN = v/35.0; tN = t/60.0
        eff = 0.70 + 0.12*vN - 0.05*(tN-0.3)*(tN-0.3)
        eff = eff * float(self.p.regen_eff_scale or self.p.drive_eff_scale)
        return ca.fmin(0.95, ca.fmax(0.40, eff))
    mode = self._select_mode(float(v_ms), float(abs(tau_nm)))
    v_grid, t_grid, Z = self.maps_regen.get(mode, self.maps_regen['default'])
    eff = float(bilinear_interp(v_grid, t_grid, Z, float(v_ms), float(abs(tau_nm))))
    eff *= float(self.p.regen_eff_scale or self.p.drive_eff_scale)
    return float(np.clip(eff, 0.40, 0.95))
```

7.7 L150 関数 SolarCarModel._update_mode_limits

- 行範囲: L150–L156
- 所属: SolarCarModel
- 定義: `_update_mode_limits(self)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `items`, `max`
- 戻り値の要点: 戻り値が無いか、`return` 文が明示されていない。
- 主なローカル代入名: `_`, `k`, `t_grid`
- 読み取る主なインスタンス属性: `self.maps_drive`, `self.tau_max`
- 更新する主なインスタンス属性: `self.tau_max`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 1、`try` 1

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. `self.tau_max` に `{}` の結果を代入する。
2. `self.maps_drive.items()` を順に走査し、各要素を `(k, (_, t_grid, _))` に入れて処理する。
3. 例外処理を伴う `try` ブロックを実行する。
4. `self.tau_max[k]` に `float(max(t_grid))` の結果を代入する。
5. `Exception` を捕捉した場合:
6. `self.tau_max[k]` に `0.0` の結果を代入する。

代表コード断片

```
def _update_mode_limits(self):
    self.tau_max = {}
    for k, (_, t_grid, _) in self.maps_drive.items():
        try:
            self.tau_max[k] = float(max(t_grid))
        except Exception:
            self.tau_max[k] = 0.0
```

7.8 L158 関数 SolarCarModel._select_mode

- 行範囲: L158–L167
- 所属: SolarCarModel
- 定義: `_select_mode(self, v_ms: float, tau_nm: float) -> str`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `get`, `lower`, `str`
- 戻り値の要点: `'eco' if 'eco' in self.maps_drive else 'default' / mode / 'power' if 'power' in self.maps_drive else 'default'`
- 主なローカル代入名: `eco_max`, `margin`, `mode`
- 読み取る主なインスタンス属性: `self.drive_mode`, `self.drive_mode_tau_margin`, `self.maps_drive`, `self.tau_max`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `mode` に `str(self.drive_mode or 'default').lower()` の結果を代入する。
2. 条件 `mode in ('eco', 'power')` を判定し、真なら内部処理を行う。
3. `mode` を返す。
4. `eco_max` に `self.tau_max.get('eco', self.tau_max.get('default', 0.0))` の結果を代入する。
5. `margin` に `float(self.drive_mode_tau_margin or 0.0)` の結果を代入する。
6. 条件 `tau_nm > eco_max + margin` を判定し、真なら内部処理を行う。
7. `'power' if 'power' in self.maps_drive else 'default'` を返す。
8. `'eco' if 'eco' in self.maps_drive else 'default'` を返す。

代表コード断片

```
def _select_mode(self, v_ms: float, tau_nm: float) -> str:
    mode = str(self.drive_mode or 'default').lower()
    if mode in ('eco', 'power'):
        return mode
    # auto
    eco_max = self.tau_max.get('eco', self.tau_max.get('default', 0.0))
    margin = float(self.drive_mode_tau_margin or 0.0)
    if tau_nm > (eco_max + margin):
        return 'power' if 'power' in self.maps_drive else 'default'
    return 'eco' if 'eco' in self.maps_drive else 'default'
```

7.9 L169 関数 SolarCarModel.select_drive_mode

- 行範囲: L169–L170
- 所属: SolarCarModel
- 定義: `select_drive_mode(self, v_ms: float, tau_nm: float) -> str`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_select_mode`, `abs`
- 戻り値の要点: `self._select_mode(v_ms, abs(tau_nm))`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self._select_mode`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘>’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `self._select_mode(v_ms, abs(tau_nm))` を返す。

代表コード断片

```
def select_drive_mode(self, v_ms: float, tau_nm: float) -> str:
    return self._select_mode(v_ms, abs(tau_nm))
```

7.10 L172 関数 SolarCarModel.R_int

- 行範囲: L172–L177
- 所属: SolarCarModel
- 定義: `R_int(self, T_C, z)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_is_symbolic`, `bilinear_interp`, `float`
- 戻り値の要点: `(R0+R_T+R_z) * float(self.p.rint_scale) / float(self.p.rint_scale) * float(bilinear_interp(self.Tg, self.zg, self.Rmap, T_C, z))`
- 主なローカル代入名: `R0`, `R_T`, `R_z`
- 読み取る主なインスタンス属性: `self.Rmap`, `self.Tg`, `self.p`, `self.zg`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. 条件 `_is_symbolic(T_C) or _is_symbolic(z)` を判定し、真なら内部処理を行う。
2. `R0` に `0.015` の結果を代入する。
3. `R_T` に `0.0002 * (25.0 - T_C)` の結果を代入する。
4. `R_z` に `0.01 * (1.0 - z)` の結果を代入する。
5. `(R0 + R_T + R_z) * float(self.p.rint_scale)` を返す。
6. 上の条件が偽の場合:
7. `float(self.p.rint_scale) * float(bilinear_interp(self.Tg, self.zg, self.Rmap, T_C, z))` を返す。

代表コード断片

```
def R_int(self, T_C, z):
    if _is_symbolic(T_C) or _is_symbolic(z):
        R0=0.015; R_T=0.0002*(25.0-T_C); R_z=0.01*(1.0-z)
        return (R0+R_T+R_z) * float(self.p.rint_scale)
    else:
        return float(self.p.rint_scale) * float(bilinear_interp(self.Tg, self.zg, self.
Rmap, T_C, z))
```

7.11 L179 関数 SolarCarModel.pv_balance

- 行範囲: L179–L220
- 所属: SolarCarModel
- 定義: `pv_balance(self, G_poa, T_cell_C)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_is_symbolic`, `bilinear_interp`, `dict`, `float`, `fmax`, `fmin`, `max`, `min`
- 戻り値の要点: `dict(eta_panel=eta_panel, eta_mppt=eta_mppt, P_pv_raw=P_pv_raw, P_pv_unlimited=P_pv_unlimited, P_pv_limit_loss=P_pv_unlimited - P_pv, P_pv=P_pv)`
- 主なローカル代入名: `P_pv`, `P_pv_raw`, `P_pv_unlimited`, `eta_mppt`, `eta_panel`, `power_limit_w`, `symbolic`
- 読み取る主なインスタンス属性: `self.Gg`, `self.Gm`, `self.Tcg`, `self.Tm`, `self.Z_mppt`, `self.Z_panel`, `self.mppt_eff_map`, `self.p`, `self.panel_eff_map`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 6、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. symbolic に `_is_symbolic(G_poa)` or `_is_symbolic(T_cell_C)` の結果を代入する。
2. 条件 symbolic を判定し、真なら内部処理を行う。
3. eta_panel に `self.p.pv_eta_ref * (1.0 + self.p.pv_mu_p * (T_cell_C - 25.0))` の結果を代入する。
4. eta_panel に `ca.fmax(0.0, eta_panel)` の結果を代入する。
5. 上の条件が偽の場合:
6. 条件 `self.panel_eff_map` を判定し、真なら内部処理を行う。
7. eta_panel に `bilinear_interp(self.Gg, self.Tcg, self.Z_panel, float(G_poa), float(T_cell_C))` の結果を代入する。
8. eta_panel に `max(0.0, float(eta_panel))` の結果を代入する。
9. 上の条件が偽の場合:
10. eta_panel に `self.p.pv_eta_ref * (1.0 + self.p.pv_mu_p * (T_cell_C - 25.0))` の結果を代入する。
11. eta_panel に `max(0.0, float(eta_panel))` の結果を代入する。
12. eta_panel を Mult で更新する。
13. P_pv_raw に `eta_panel * self.p.pv_area * G_poa` の結果を代入する。
14. 条件 symbolic を判定し、真なら内部処理を行う。
15. eta_mppt に `float(self.p.mppt_eta)` の結果を代入する。
16. 上の条件が偽の場合:
17. 条件 `self.mppt_eff_map` を判定し、真なら内部処理を行う。
18. eta_mppt に `bilinear_interp(self.Gm, self.Tm, self.Z_mppt, float(G_poa), float(T_cell_C))` の結果を代入する。
19. eta_mppt に `max(0.0, float(eta_mppt))` の結果を代入する。
20. 上の条件が偽の場合:
21. eta_mppt に `float(self.p.mppt_eta)` の結果を代入する。
22. P_pv_unlimited に `eta_mppt * P_pv_raw` の結果を代入する。
23. power_limit_w に `max(0.0, float(self.p.pv_power_limit_w))` の結果を代入する。
24. 条件 `power_limit_w > 0.0` を判定し、真なら内部処理を行う。
25. 条件 symbolic を判定し、真なら内部処理を行う。
26. P_pv に `ca.fmin(P_pv_unlimited, power_limit_w)` の結果を代入する。
27. 上の条件が偽の場合:
28. P_pv に `min(float(P_pv_unlimited), power_limit_w)` の結果を代入する。
29. 上の条件が偽の場合:
30. P_pv に P_pv_unlimited の結果を代入する。
31. `dict(eta_panel=eta_panel, eta_mppt=eta_mppt, P_pv_raw=P_pv_raw, P_pv_unlimited=P_pv_unlimited, P_pv_limit_loss=P_pv_unlimited - P_pv, P_pv=P_pv)` を返す。

代表コード断片

```
def pv_balance(self, G_poa, T_cell_C):
    symbolic = _is_symbolic(G_poa) or _is_symbolic(T_cell_C)
    if symbolic:
        # Map interpolation is numeric. Keep the symbolic optimization
        # path differentiable by using the map's calibrated reference
        # model, as is done for the symbolic drive-efficiency path.
        eta_panel = self.p.pv_eta_ref * (
            1.0 + self.p.pv_mu_p * (T_cell_C - 25.0)
        )
        eta_panel = ca.fmax(0.0, eta_panel)
    elif self.panel_eff_map:
        eta_panel = bilinear_interp(self.Gg, self.Tcg, self.Z_panel, float(G_poa), float(
            T_cell_C))
        eta_panel = max(0.0, float(eta_panel))
    else:
        eta_panel = self.p.pv_eta_ref*(1.0+self.p.pv_mu_p*(T_cell_C-25.0))
        eta_panel = max(0.0, float(eta_panel))
    eta_panel *= float(self.p.panel_gain)
    P_pv_raw = eta_panel*self.p.pv_area*G_poa
    if symbolic:
        eta_mppt = float(self.p.mppt_eta)
    elif self.mppt_eff_map:
        eta_mppt = bilinear_interp(self.Gm, self.Tm, self.Z_mppt, float(G_poa), float(
            T_cell_C))
        eta_mppt = max(0.0, float(eta_mppt))
    else:
        eta_mppt = float(self.p.mppt_eta)
    P_pv_unlimited = eta_mppt*P_pv_raw
    power_limit_w = max(0.0, float(self.p.pv_power_limit_w))
    if power_limit_w > 0.0:
        if symbolic:
            P_pv = ca.fmin(P_pv_unlimited, power_limit_w)
        else:
            P_pv = min(float(P_pv_unlimited), power_limit_w)
    else:
        P_pv = P_pv_unlimited
    return dict(
...

```

7.12 L222 関数 SolarCarModel.pv_power_mppt

- 行範囲: L222–L223
- 所属: SolarCarModel
- 定義: `pv_power_mppt(self, G_poa, T_cell_C)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `pv_balance`
- 戻り値の要点: `self.pv_balance(G_poa, T_cell_C)['P_pv']`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self.pv_balance`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. self.pv_balance(G_poa, T_cell_C)['P_pv'] を返す。

代表コード断片

```
def pv_power_mppt(self, G_poa, T_cell_C):  
    return self.pv_balance(G_poa, T_cell_C)['P_pv']
```

7.13 L225 関数 SolarCarModel._scaled_slope_pct

- 行範囲: L225–L226
- 所属: SolarCarModel
- 定義: _scaled_slope_pct(self, slope_pct)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: float
- 戻り値の要点: slope_pct * float(self.p.grade_scale)
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: self.p
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. slope_pct * float(self.p.grade_scale) を返す。

代表コード断片

```
def _scaled_slope_pct(self, slope_pct):  
    return slope_pct * float(self.p.grade_scale)
```

7.14 L228 関数 SolarCarModel.charge_efficiency

- 行範囲: L228–L233
- 所属: SolarCarModel
- 定義: charge_efficiency(self, P_pack)->float
- docstring: 明示されていない。

- このブロックが直接呼ぶ主な関数・メソッド: float
- 戻り値の要点: `float(self.p.eta_charge) if p_pack < 0.0 else 1.0 / 1.0`
- 主なローカル代入名: `p_pack`
- 読み取る主なインスタンス属性: `self.p`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 1

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 例外処理を伴う `try` ブロックを実行する。
2. `p_pack` に `float(P_pack)` の結果を代入する。
3. `Exception` を捕捉した場合:
4. 1.0 を返す。
5. `float(self.p.eta_charge) if p_pack < 0.0 else 1.0` を返す。

代表コード断片

```
def charge_efficiency(self, P_pack) -> float:
    try:
        p_pack = float(P_pack)
    except Exception:
        return 1.0
    return float(self.p.eta_charge) if p_pack < 0.0 else 1.0
```

7.15 L235 関数 `SolarCarModel.soc_step`

- 行範囲: L235–L260
- 所属: `SolarCarModel`
- 定義: `soc_step(self, z:float, P_pack:float, dt_sec:float, *, current_a:float|None=None, Tbat_C:float=25.0)->float`
- docstring: Advance SoC using the profile's declared state definition.
A positive “`Q_nom_Ah`” selects charge SoC and therefore coulomb counting, which is consistent with an OCV-vs-SoC map. Legacy profiles that do not declare pack capacity retain energy integration so existing non-solar profiles are not silently reinterpreted.
- このブロックが直接呼ぶ主な関数・メソッド: `battery_iv`, `charge_efficiency`, float, max
- 戻り値の要点: `float(z) - eta * (float(P_pack) * float(dt_sec) / 3600.0) / max(float(self.p.E_nom_Wh), 1e-06) / float(z) - eta * current * float(dt_sec) / (3600.0 * q_nom_ah)`

- 主なローカル代入名: `current`, `eta`, `q_nom_ah`
- 読み取る主なインスタンス属性: `self.battery_iv`, `self.charge_efficiency`, `self.p`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `eta` に `self.charge_efficiency(P_pack)` の結果を代入する。
2. `q_nom_ah` に `float(self.p.Q_nom_Ah)` の結果を代入する。
3. 条件 `q_nom_ah > 0.0` を判定し、真なら内部処理を行う。
4. `current` に `float(current_a) if current_a is not None else float(self.battery_iv(P_pack, z, Tbat_C)['I'])` の結果を代入する。
5. `float(z) - eta * current * float(dt_sec) / (3600.0 * q_nom_ah)` を返す。
6. `float(z) - eta * (float(P_pack) * float(dt_sec) / 3600.0) / max(float(self.p.E_nom_Wh), 1e-06)` を返す。

代表コード断片

```
def soc_step(
    self,
    z: float,
    P_pack: float,
    dt_sec: float,
    *,
    current_a: float | None = None,
    Tbat_C: float = 25.0,
) -> float:
    """Advance SoC using the profile's declared state definition.

    A positive ``Q_nom_Ah`` selects charge SoC and therefore coulomb
    counting, which is consistent with an OCV-vs-SoC map. Legacy
    profiles that do not declare pack capacity retain energy integration
    so existing non-solar profiles are not silently reinterpreted.
    """
    eta = self.charge_efficiency(P_pack)
    q_nom_ah = float(self.p.Q_nom_Ah)
    if q_nom_ah > 0.0:
        current = (
            float(current_a)
            if current_a is not None
            else float(self.battery_iv(P_pack, z, Tbat_C)["I"])
        )
        return float(z) - eta * current * float(dt_sec) / (3600.0 * q_nom_ah)
    return float(z) - eta * (float(P_pack) * float(dt_sec) / 3600.0) / max(float(self.p.
E_nom_Wh), 1.0e-6)
```

7.16 L262 関数 SolarCarModel.ocv_from_soc

- 行範囲: L262–L269
- 所属: SolarCarModel
- 定義: `ocv_from_soc(self, z)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_is_symbolic`, `clip`, `float`, `fmax`, `fmin`, `interp`
- 戻り値の要点: `float(np.interp(zc, self.soc_grid, self.ocv_grid)) / self.p.V_min + (self.p.V_max - self.p.V_min) * z_clamped / self.p.V_min + (self.p.V_max - self.p.V_min) * zc`
- 主なローカル代入名: `z_clamped`, `zc`
- 読み取る主なインスタンス属性: `self.ocv_grid`, `self.ocv_soc_map`, `self.p`, `self.soc_grid`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. 条件 `_is_symbolic(z)` を判定し、真なら内部処理を行う。
2. `z_clamped` に `ca.fmin(self.p.soc_max, ca.fmax(self.p.soc_min, z))` の結果を代入する。
3. `self.p.V_min + (self.p.V_max - self.p.V_min) * z_clamped` を返す。
4. `zc` に `float(np.clip(z, self.p.soc_min, self.p.soc_max))` の結果を代入する。
5. 条件 `not self.ocv_soc_map` を判定し、真なら内部処理を行う。
6. `self.p.V_min + (self.p.V_max - self.p.V_min) * zc` を返す。
7. `float(np.interp(zc, self.soc_grid, self.ocv_grid))` を返す。

代表コード断片

```
def ocv_from_soc(self, z):
    if _is_symbolic(z):
        z_clamped = ca.fmin(self.p.soc_max, ca.fmax(self.p.soc_min, z))
        return self.p.V_min + (self.p.V_max - self.p.V_min) * z_clamped
    zc = float(np.clip(z, self.p.soc_min, self.p.soc_max))
    if not self.ocv_soc_map:
        return self.p.V_min + (self.p.V_max - self.p.V_min) * zc
    return float(np.interp(zc, self.soc_grid, self.ocv_grid))
```

7.17 L271 関数 SolarCarModel.load_ocv_map

- 行範囲: L271–L278
- 所属: SolarCarModel
- 定義: load_ocv_map(self, path:str) -> bool
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: read_1d_map
- 戻り値の要点: True / False
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: self.ocv_grid, self.ocv_soc_map, self.soc_grid
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 1

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 例外処理を伴う try ブロックを実行する。
2. (self.soc_grid, self.ocv_grid) に read_1d_map(path) の結果を代入する。
3. self.ocv_soc_map に True の結果を代入する。
4. True を返す。
5. Exception を捕捉した場合:
6. self.ocv_soc_map に None の結果を代入する。
7. False を返す。

代表コード断片

```
def load_ocv_map(self, path: str) -> bool:
    try:
        self.soc_grid, self.ocv_grid = read_1d_map(path)
        self.ocv_soc_map = True
        return True
    except Exception:
        self.ocv_soc_map = None
        return False
```

7.18 L280 関数 SolarCarModel.air_density

- 行範囲: L280–L295
- 所属: SolarCarModel
- 定義: `air_density(self, ambient_temp_c=None, elevation_m=0.0)`
- docstring: Return dry-air density from the configured atmosphere model.
The ideal-gas-altitude mode uses the ISA tropospheric pressure law and measured ambient temperature. “rho” remains the explicit fallback so old profiles retain bit-for-bit constant-density behavior.
- このブロックが直接呼ぶ主な関数・メソッド: `clip`, `float`, `lower`, `max`, `str`
- 戻り値の要点: `float(pressure_pa / (287.05 * temperature_k)) / float(self.p.rho) / float(self.p.rho)`
- 主なローカル代入名: `altitude`, `pressure_pa`, `pressure_ratio`, `temperature_k`
- 読み取る主なインスタンス属性: `self.p`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. 条件 `str(self.p.air_density_mode or 'constant').lower() != 'ideal_gas_altitude'` を判定し、真なら内部処理を行う。
2. `float(self.p.rho)` を返す。
3. 条件 `ambient_temp_c is None` を判定し、真なら内部処理を行う。
4. `float(self.p.rho)` を返す。
5. `temperature_k` に `max(180.0, float(ambient_temp_c) + 273.15)` の結果を代入する。
6. `altitude` に `float(np.clip(elevation_m, -500.0, 11000.0))` の結果を代入する。
7. `pressure_ratio` に `max(0.05, 1.0 - 0.0065 * altitude / 288.15) ** 5.255877` の結果を代入する。
8. `pressure_pa` に `max(1000.0, float(self.p.air_density_reference_pressure_pa)) * pressure_ratio` の結果を代入する。
9. `float(pressure_pa / (287.05 * temperature_k))` を返す。

代表コード断片

```
def air_density(self, ambient_temp_c=None, elevation_m=0.0):
    """Return dry-air density from the configured atmosphere model.

    The ideal-gas-altitude mode uses the ISA tropospheric pressure law and
    measured ambient temperature. ``rho`` remains the explicit fallback so
    old profiles retain bit-for-bit constant-density behavior.
    """
    if str(self.p.air_density_mode or 'constant').lower() != 'ideal_gas_altitude':
        return float(self.p.rho)
    if ambient_temp_c is None:
        return float(self.p.rho)
    temperature_k = max(180.0, float(ambient_temp_c) + 273.15)
    altitude = float(np.clip(elevation_m, -500.0, 11000.0))
    pressure_ratio = max(0.05, 1.0 - 0.0065 * altitude / 288.15) ** 5.255877
    pressure_pa = max(1000.0, float(self.p.air_density_reference_pressure_pa)) *
    pressure_ratio
    return float(pressure_pa / (287.05 * temperature_k))
```

7.19 L297 関数 SolarCarModel.resistive_forces

- 行範囲: L297-L342
- 所属: SolarCarModel
- 定義: `resistive_forces(self, v_ms, slope_pct, headwind_ms=0.0, *, ambient_temp_c=None, elevation_m=0.0, air_density_kgm3=None)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_is_symbolic`, `_scaled_slope_pct`, `air_density`, `atan`, `cos`, `dict`, `float`, `fmax`, `max`, `sin`
- 戻り値の要点: `dict(F_aero=F_aero, F_roll=F_roll, F_grade=F_grade, F_total=F_total, theta=theta, v_rel_ms=v_rel, air_density_kgm3=float(rho), normal_force_n=N, Crr_eff=Crr_eff, slope_scaled_pct=float(self._scaled_slope_pct) / dict(F_aero=F_aero, F_roll=F_roll, F_grade=F_grade, F_total=F_total, theta=theta, v_rel_ms=v_rel, air_density_kgm3=rho, normal_force_n=N, Crr_eff=Crr_eff, slope_scaled_pct=self._scaled_slope_pct(slope_pct)))`
- 主なローカル代入名: `Crr_eff`, `F_aero`, `F_grade`, `F_roll`, `F_total`, `N`, `rho`, `theta`, `v_rel`
- 読み取る主なインスタンス属性: `self._scaled_slope_pct`, `self.air_density`, `self.p`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `rho` に `self.air_density(ambient_temp_c, elevation_m)` if `air_density_kgm3` is `None` else `air_density_kgm3` の結果を代入する。
2. 条件 `_is_symbolic(v_ms)` or `_is_symbolic(slope_pct)` or `_is_symbolic(headwind_ms)` を判定し、真なら内部処理を行う。

3. `v_rel` に `ca.fmax(0.0, v_ms + headwind_ms)` の結果を代入する。
4. `theta` に `ca.atan(self._scaled_slope_pct(slope_pct) / 100.0)` の結果を代入する。
5. `F_aero` に `0.5 * rho * self.p.CdA * v_rel ** 2` の結果を代入する。
6. `N` に `self.p.m * self.p.g * ca.cos(theta)` の結果を代入する。
7. `Crr_eff` に `self.p.Crr` の結果を代入する。
8. 条件 `self.p.Crr_per_wheel and self.p.wheel_count` を判定し、真なら内部処理を行う。
9. `Crr_eff` に `self.p.Crr_per_wheel * float(self.p.wheel_count)` の結果を代入する。
10. `F_roll` に `Crr_eff * N` の結果を代入する。
11. `F_grade` に `self.p.m * self.p.g * ca.sin(theta)` の結果を代入する。
12. `F_total` に `F_aero + F_roll + F_grade` の結果を代入する。
13. `dict(F_aero=F_aero, F_roll=F_roll, F_grade=F_grade, F_total=F_total, theta=theta, v_rel_ms=v_rel, air_density_kgm3=rho, normal_force_n=N, Crr_eff=Crr_eff, slope_scaled_pct=self._scaled_slope_pct(slope_pct))` を返す。
14. `v_rel` に `max(0.0, float(v_ms) + float(headwind_ms))` の結果を代入する。
15. `theta` に `math.atan(float(self._scaled_slope_pct(slope_pct)) / 100.0)` の結果を代入する。
16. `F_aero` に `0.5 * float(rho) * self.p.CdA * v_rel ** 2` の結果を代入する。
17. `N` に `self.p.m * self.p.g * math.cos(theta)` の結果を代入する。
18. `Crr_eff` に `self.p.Crr` の結果を代入する。
19. 条件 `self.p.Crr_per_wheel and self.p.wheel_count` を判定し、真なら内部処理を行う。
20. `Crr_eff` に `self.p.Crr_per_wheel * float(self.p.wheel_count)` の結果を代入する。
21. `F_roll` に `Crr_eff * N` の結果を代入する。
22. `F_grade` に `self.p.m * self.p.g * math.sin(theta)` の結果を代入する。
23. `F_total` に `F_aero + F_roll + F_grade` の結果を代入する。
24. `dict(F_aero=F_aero, F_roll=F_roll, F_grade=F_grade, F_total=F_total, theta=theta, v_rel_ms=v_rel, air_density_kgm3=float(rho), normal_force_n=N, Crr_eff=Crr_eff, slope_scaled_pct=float(self._scaled_slope_pct(slope_pct)))` を返す。

代表コード断片

```
def resistive_forces(
    self,
    v_ms,
    slope_pct,
    headwind_ms=0.0,
    *,
    ambient_temp_c=None,
    elevation_m=0.0,
    air_density_kgm3=None,
):
    rho = (
        self.air_density(ambient_temp_c, elevation_m)
        if air_density_kgm3 is None
        else air_density_kgm3
    )
    if _is_symbolic(v_ms) or _is_symbolic(slope_pct) or _is_symbolic(headwind_ms):
```



```

v_rel = ca.fmax(0.0, v_ms + headwind_ms)
theta = ca.atan(self._scaled_slope_pct(slope_pct) / 100.0)
F_aero = 0.5 * rho * self.p.CdA * v_rel ** 2
N = self.p.m * self.p.g * ca.cos(theta)
Crr_eff = self.p.Crr
if self.p.Crr_per_wheel and self.p.wheel_count:
    Crr_eff = self.p.Crr_per_wheel * float(self.p.wheel_count)
F_roll = Crr_eff * N
F_grade = self.p.m * self.p.g * ca.sin(theta)
F_total = F_aero + F_roll + F_grade
return dict(F_aero=F_aero, F_roll=F_roll, F_grade=F_grade,
            F_total=F_total, theta=theta, v_rel_ms=v_rel,
            air_density_kgm3=rho,
            normal_force_n=N, Crr_eff=Crr_eff,
            slope_scaled_pct=self._scaled_slope_pct(slope_pct))
v_rel = max(0.0, float(v_ms) + float(headwind_ms))
theta = math.atan(float(self._scaled_slope_pct(slope_pct)) / 100.0)
F_aero = 0.5 * float(rho) * self.p.CdA * v_rel ** 2
N = self.p.m * self.p.g * math.cos(theta)
...

```

7.20 L344 関数 SolarCarModel.battery_iv

- 行範囲: L344–L366
- 所属: SolarCarModel
- 定義: battery_iv(self, P_pack, z, Tbat_C)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: R_int, _is_symbolic, clip, dict, float, fmax, fmin, max, ocv_from_soc, sqrt
- 戻り値の要点: dict(I=I, V=V, OCV=OCV, Rint=Rint, Rline=Rline, Rpolarization=Rpolarization, Rtotal=Rtot, iv_discriminant=disc)
- 主なローカル代入名: I, OCV, Rint, Rline, Rpolarization, Rtot, V, a, b, c, disc, symbolic, z_rint
- 読み取る主なインスタンス属性: self.R_int, self.ocv_from_soc, self.p
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. OCV に self.ocv_from_soc(z) の結果を代入する。
2. symbolic に _is_symbolic(P_pack) or _is_symbolic(z) or _is_symbolic(Tbat_C) の結果を代入する。
3. z_rint に ca.fmin(0.95, ca.fmax(0.1, z)) if symbolic else float(np.clip(z, 0.1, 0.95)) の結果を代入する。
4. Rint に self.R_int(Tbat_C, z_rint) の結果を代入する。

5. Rline に `float(self.p.r_line_ohm)` の結果を代入する。
6. Rpolarization に `max(0.0, float(self.p.r_polarization_ohm))` の結果を代入する。
7. Rtot に `Rint + Rline + Rpolarization` の結果を代入する。
8. a に Rtot の結果を代入する。
9. b に `-OCV` の結果を代入する。
10. c に `P_pack` の結果を代入する。
11. 条件 `symbolic` を判定し、真なら内部処理を行う。
12. disc に `ca.fmax(b * b - 4 * a * c, 0.0)` の結果を代入する。
13. I に `(OCV - ca.sqrt(disc)) / (2 * Rtot)` の結果を代入する。
14. 上の条件が偽の場合:
15. disc に `max(float(b * b - 4 * a * c), 0.0)` の結果を代入する。
16. I に `(float(OCV) - math.sqrt(disc)) / (2 * float(Rtot))` の結果を代入する。
17. V に `OCV - I * Rtot` の結果を代入する。
18. `dict(I=I, V=V, OCV=OCV, Rint=Rint, Rline=Rline, Rpolarization=Rpolarization, Rtotal=Rtot, iv_discriminant=disc)` を返す。

代表コード断片

```
def battery_iv(self, P_pack, z, Tbat_C):
    OCV = self.ocv_from_soc(z)
    symbolic = _is_symbolic(P_pack) or _is_symbolic(z) or _is_symbolic(Tbat_C)
    z_rint = ca.fmin(0.95, ca.fmax(0.1, z)) if symbolic else float(np.clip(z, 0.1, 0.95))
    )
    Rint = self.R_int(Tbat_C, z_rint)
    Rline = float(self.p.r_line_ohm)
    Rpolarization = max(0.0, float(self.p.r_polarization_ohm))
    # Planner intervals are much longer than the fitted polarization time
    # constant, so the stateless MPC model uses its steady-state resistance.
    Rtot = Rint + Rline + Rpolarization
    a = Rtot
    b = -OCV
    c = P_pack
    if symbolic:
        disc = ca.fmax(b * b - 4 * a * c, 0.0)
        I = (OCV - ca.sqrt(disc)) / (2 * Rtot)
    else:
        disc = max(float(b * b - 4 * a * c), 0.0)
        I = (float(OCV) - math.sqrt(disc)) / (2 * float(Rtot))
    V = OCV - I * Rtot
    return dict(I=I, V=V, OCV=OCV, Rint=Rint, Rline=Rline,
                Rpolarization=Rpolarization,
                Rtotal=Rtot, iv_discriminant=disc)
```

7.21 L368 関数 SolarCarModel.mech_power

- 行範囲: L368–L387
- 所属: SolarCarModel
- 定義: `mech_power(self, v_ms, slope_pct, headwind_ms=0.0, inertial_power_w=0.0, *, ambient_temp_c=None, elevation_m=0.0)`

- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `resistive_forces`
- 戻り値の要点: `forces['F_total'] * v_ms + inertial_power_w`
- 主なローカル代入名: `forces`
- 読み取る主なインスタンス属性: `self.resistive_forces`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `forces` に `self.resistive_forces(v_ms, slope_pct, headwind_ms, ambient_temp_c=ambient_temp_c, elevation_m=elevation_m)` の結果を代入する。
2. `forces['F_total'] * v_ms + inertial_power_w` を返す。

代表コード断片

```
def mech_power(
    self,
    v_ms,
    slope_pct,
    headwind_ms=0.0,
    inertial_power_w=0.0,
    *,
    ambient_temp_c=None,
    elevation_m=0.0,
):
    forces = self.resistive_forces(
        v_ms,
        slope_pct,
        headwind_ms,
        ambient_temp_c=ambient_temp_c,
        elevation_m=elevation_m,
    )
    # Aerodynamic force depends on air-relative speed, while wheel work is
    # force times ground speed. This matches the identification model.
    return forces['F_total'] * v_ms + inertial_power_w
```

7.22 L389 関数 `SolarCarModel.auxiliary_power`

- 行範囲: L389–L394
- 所属: `SolarCarModel`
- 定義: `auxiliary_power(self, aux_power_w=None)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `max`

- 戻り値の要点: `max(0.0, float(self.p.P_aux)) / max(0.0, float(aux_power_w)) / max(0.0, float(self.aux_power_override_w))`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self.aux_power_override_w`, `self.p`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. 条件 `aux_power_w is not None` を判定し、真なら内部処理を行う。
2. `max(0.0, float(aux_power_w))` を返す。
3. 条件 `self.aux_power_override_w is not None` を判定し、真なら内部処理を行う。
4. `max(0.0, float(self.aux_power_override_w))` を返す。
5. `max(0.0, float(self.p.P_aux))` を返す。

代表コード断片

```
def auxiliary_power(self, aux_power_w=None):
    if aux_power_w is not None:
        return max(0.0, float(aux_power_w))
    if self.aux_power_override_w is not None:
        return max(0.0, float(self.aux_power_override_w))
    return max(0.0, float(self.p.P_aux))
```

7.23 L396 関数 `SolarCarModel.scheduled_auxiliary_power`

- 行範囲: L396–L403
- 所属: `SolarCarModel`
- 定義: `scheduled_auxiliary_power(self, *, is_driving: bool, irradiance_wm2: float) -> float`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `auxiliary_power`, `float`, `max`
- 戻り値の要点: `max(0.0, float(self.p.P_aux_stopped)) / self.auxiliary_power() / max(0.0, float(self.p.P_aux_night)) / self.auxiliary_power()`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self.aux_power_override_w`, `self.auxiliary_power`, `self.p`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `is_driving` を判定し、真なら内部処理を行う。
2. `self.auxiliary_power()` を返す。
3. 条件 `float(irradiance_wm2) <= max(0.0, float(self.p.aux_night_ghi_threshold_wm2))` を判定し、真なら内部処理を行う。
4. `max(0.0, float(self.p.P_aux_night))` を返す。
5. 条件 `self.aux_power_override_w is not None` を判定し、真なら内部処理を行う。
6. `self.auxiliary_power()` を返す。
7. `max(0.0, float(self.p.P_aux_stopped))` を返す。

代表コード断片

```
def scheduled_auxiliary_power(self, *, is_driving: bool, irradiance_wm2: float) -> float:
    :
    if is_driving:
        return self.auxiliary_power()
    if float(irradiance_wm2) <= max(0.0, float(self.p.aux_night_ghi_threshold_wm2)):
        return max(0.0, float(self.p.P_aux_night))
    if self.aux_power_override_w is not None:
        return self.auxiliary_power()
    return max(0.0, float(self.p.P_aux_stopped))
```

7.24 L405 関数 `SolarCarModel.torque_from_mech`

- 行範囲: L405–L414
- 所属: `SolarCarModel`
- 定義: `torque_from_mech(self, P_mech, v_ms, wheel_radius=None, ratio=None)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: $(T_m, \omega_w * ratio)$
- 主なローカル代入名: `T_m`, `T_w`, `eps`, `omega_w`, `ratio`, `wheel_radius`
- 読み取る主なインスタンス属性: `self.p`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、try 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. 条件 `wheel_radius is None` を判定し、真なら内部処理を行う。
2. `wheel_radius` に `self.p.wheel_radius` の結果を代入する。
3. 条件 `ratio is None` を判定し、真なら内部処理を行う。
4. `ratio` に `self.p.gear_ratio` の結果を代入する。
5. `eps` に 0.001 の結果を代入する。
6. `omega_w` に `v_ms / wheel_radius` の結果を代入する。
7. `T_w` に `P_mech / (omega_w + eps)` の結果を代入する。
8. `T_m` に `T_w / ratio` の結果を代入する。
9. `(T_m, omega_w * ratio)` を返す。

代表コード断片

```
def torque_from_mech(self, P_mech, v_ms, wheel_radius=None, ratio=None):
    if wheel_radius is None:
        wheel_radius = self.p.wheel_radius
    if ratio is None:
        ratio = self.p.gear_ratio
    eps=1e-3
    omega_w = v_ms/wheel_radius
    T_w = P_mech/(omega_w+eps)
    T_m = T_w/ratio
    return T_m, omega_w*ratio
```

7.25 L416 関数 `SolarCarModel.electrical_balance`

- 行範囲: L416–L477
- 所属: `SolarCarModel`
- 定義: `electrical_balance(self, v_ms, slope_pct, z, Tbat_C, G_poa, Tcell_C, headwind_ms=0.0, aux_power_w=None, inertial_power_w=0.0, ambient_temp_c=None, elevation_m=0.0)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_is_symbolic`, `auxiliary_power`, `battery_iv`, `charge_efficiency`, `clip`, `dict`, `eff_drive`, `eff_regen`, `float`, `fmax`, `max`, `pv_balance`
- 戻り値の要点: `dict(P_pv=P_pv, P_pv_raw=pv['P_pv_raw'], P_pv_unlimited=pv['P_pv_unlimited'], P_pv_limit_loss=pv['P_pv_limit_loss'], eta_panel=pv['eta_panel'], eta_mppt=pv['eta_mppt'], P_mech=P_mech_wheel, P_mech_wheel=P_mech_wheel, P_road_load=P_road_load, P_inertia=inertial_power_w, P_aux=P_aux, P_pack=P_pack, I=I, V=V, losses_line=losses_line, losses_int=losses_int, losses_rint=losses_rint, losses_polarization=losses_polarization, OCV=OCV, Rint=Rint, Rline=Rline, Rpolarization=Rpolarization, Rtotal=Rtot, iv_discriminant=disc, eta_charge=eta_charge, P_dc_to_drv=P_dc_to_drv, P_reg_to_dc=P_reg_to_dc, eff_drv=eff_drv, eff_reg=eff_reg, torque_drive_nm=Tm_drv, torque_regen_nm=Tm_reg, omega_motor_radps=omega_m, omega_wheel_radps=omega_m / max(float(self.p.gear_ratio), 1e-09))`
- 主なローカル代入名: `I`, `OCV`, `P_aux`, `P_dc_to_drv`, `P_mech_neg`, `P_mech_pos`, `P_mech_wheel`, `P_pack`, `P_pv`, `P_reg_to_dc`, `P_road_load`, `Rint`, `Rline`, `Rpolarization`, `Rtot`, `Tm_drv`, `Tm_reg`, `V`, `_`, `disc`
- 読み取る主なインスタンス属性: `self.auxiliary_power`, `self.battery_iv`, `self.charge_efficiency`, `self.eff_drive`, `self.eff_regen`, `self.p`, `self.pv_balance`, `self.resistive_forces`, `self.torque_from_mech`

- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。

上から順の処理

1. `pv` に `self.pv_balance(G_poa, Tcell_C)` の結果を代入する。
2. `P_pv` に `pv['P_pv']` の結果を代入する。
3. `road_forces` に `self.resistive_forces(v_ms, slope_pct, headwind_ms, ambient_temp_c=ambient_temp_c, elevation_m=elevation_m)` の結果を代入する。
4. `P_road_load` に `road_forces['F_total'] * v_ms` の結果を代入する。
5. `P_mech_wheel` に `P_road_load + inertial_power_w` の結果を代入する。
6. `symbolic` に `_is_symbolic(P_mech_wheel) or _is_symbolic(z) or _is_symbolic(Tbat_C) or _is_symbolic(inertial_power_w)` の結果を代入する。
7. 条件 `symbolic` を判定し、真なら内部処理を行う。
8. `P_mech_pos` に `ca.fmax(P_mech_wheel, 0.0)` の結果を代入する。
9. `P_mech_neg` に `ca.fmax(-P_mech_wheel, 0.0)` の結果を代入する。
10. 上の条件が偽の場合:
11. `P_mech_pos` に `max(float(P_mech_wheel), 0.0)` の結果を代入する。
12. `P_mech_neg` に `max(-float(P_mech_wheel), 0.0)` の結果を代入する。
13. `(Tm_drv, omega_m)` に `self.torque_from_mech(P_mech_pos, v_ms)` の結果を代入する。
14. `eff_drv` に `self.eff_drive(v_ms, Tm_drv)` の結果を代入する。
15. `P_dc_to_drv` に `P_mech_pos / (eff_drv * self.p.gear_eta * self.p.inverter_eta)` の結果を代入する。
16. `(Tm_reg, _)` に `self.torque_from_mech(P_mech_neg, v_ms)` の結果を代入する。
17. `eff_reg` に `self.eff_regen(v_ms, Tm_reg)` の結果を代入する。
18. `regen_utilization` に `float(np.clip(self.p.regen_utilization, 0.0, 1.0))` の結果を代入する。
19. `P_reg_to_dc` に `regen_utilization * eff_reg * self.p.gear_eta * self.p.inverter_eta * P_mech_neg` の結果を代入する。
20. `P_aux` に `self.auxiliary_power(aux_power_w)` の結果を代入する。
21. `P_pack` に `P_dc_to_drv - P_reg_to_dc + P_aux - P_pv` の結果を代入する。
22. `iv` に `self.battery_iv(P_pack, z, Tbat_C)` の結果を代入する。
23. `OCV` に `iv['OCV']` の結果を代入する。
24. `Rint` に `iv['Rint']` の結果を代入する。
25. `Rline` に `iv['Rline']` の結果を代入する。

26. Rpolarization に iv['Rpolarization'] の結果を代入する。
27. Rtot に iv['Rtotal'] の結果を代入する。
28. disc に iv['iv_discriminant'] の結果を代入する。
29. I に iv['I'] の結果を代入する。
30. V に iv['V'] の結果を代入する。
31. losses_line に $I * I * R_{line}$ の結果を代入する。
32. losses_rint に $I * I * R_{int}$ の結果を代入する。
33. losses_polarization に $I * I * R_{polarization}$ の結果を代入する。
34. losses_int に $losses_rint + losses_polarization$ の結果を代入する。
35. eta_charge に self.charge_efficiency(P_pack) の結果を代入する。
36. dict(P_pv=P_pv, P_pv_raw=pv['P_pv_raw'], P_pv_unlimited=pv['P_pv_unlimited'], P_pv_limit_loss=pv['P_pv_limit_loss'], eta_panel=pv['eta_panel'], eta_mppt=pv['eta_mppt'], P_mech=P_mech_wheel, P_mech_wheel=P_mech_wheel, P_road_load=P_road_load, P_inertia=inertial_power_w, P_aux=P_aux, P_pack=P_pack, I=I, V=V, losses_line=losses_line, losses_int=losses_int, losses_rint=losses_rint, losses_polarization=losses_polarization, OCV=OCV, Rint=Rint, Rline=Rline, Rpolarization=Rpolarization, Rtotal=Rtot, iv_discriminant=disc, eta_charge=eta_charge, P_dc_to_drv=P_dc_to_drv, P_reg_to_dc=P_reg_to_dc, eff_drv=eff_drv, eff_reg=eff_reg, torque_drive_nm=Tm_drv, torque_regen_nm=Tm_reg, omega_motor_radps=omega_m, omega_wheel_radps=omega_m / max(float(self.p.gear_ratio), 1e-09)) を返す。

代表コード断片

```
def electrical_balance(self, v_ms, slope_pct, z, Tbat_C, G_poa, Tcell_C,
                      headwind_ms=0.0, aux_power_w=None, inertial_power_w=0.0,
                      ambient_temp_c=None, elevation_m=0.0):
    pv = self.pv_balance(G_poa, Tcell_C)
    P_pv = pv['P_pv']
    road_forces = self.resistive_forces(
        v_ms,
        slope_pct,
        headwind_ms,
        ambient_temp_c=ambient_temp_c,
        elevation_m=elevation_m,
    )
    P_road_load = road_forces['F_total'] * v_ms
    P_mech_wheel = P_road_load + inertial_power_w
    symbolic = (
        _is_symbolic(P_mech_wheel)
        or _is_symbolic(z)
        or _is_symbolic(Tbat_C)
        or _is_symbolic(inertial_power_w)
    )
    if symbolic:
        P_mech_pos = ca.fmax(P_mech_wheel, 0.0)
        P_mech_neg = ca.fmax(-P_mech_wheel, 0.0)
    else:
        P_mech_pos = max(float(P_mech_wheel), 0.0)
        P_mech_neg = max(-float(P_mech_wheel), 0.0)
    Tm_drv, omega_m = self.torque_from_mech(P_mech_pos, v_ms)
    eff_drv = self.eff_drive(v_ms, Tm_drv)
    P_dc_to_drv = P_mech_pos / (eff_drv * self.p.gear_eta * self.p.inverter_eta)
    Tm_reg, _ = self.torque_from_mech(P_mech_neg, v_ms)
    eff_reg = self.eff_regen(v_ms, Tm_reg)
    regen_utilization = float(np.clip(self.p.regen_utilization, 0.0, 1.0))
```



```
P_reg_to_dc = regen_utilization*eff_reg*self.p.gear_eta*self.p.inverter_eta*
P_mech_neg
P_aux = self.auxiliary_power(aux_power_w)
P_pack = P_dc_to_drv - P_reg_to_dc + P_aux - P_pv
...
```

8 処理の流れ

1. ソース中の主要関数を通じて処理を進める。

9 このファイルの位置づけ

この資料群では、`mpc_solarcar/model.py` を **model** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。